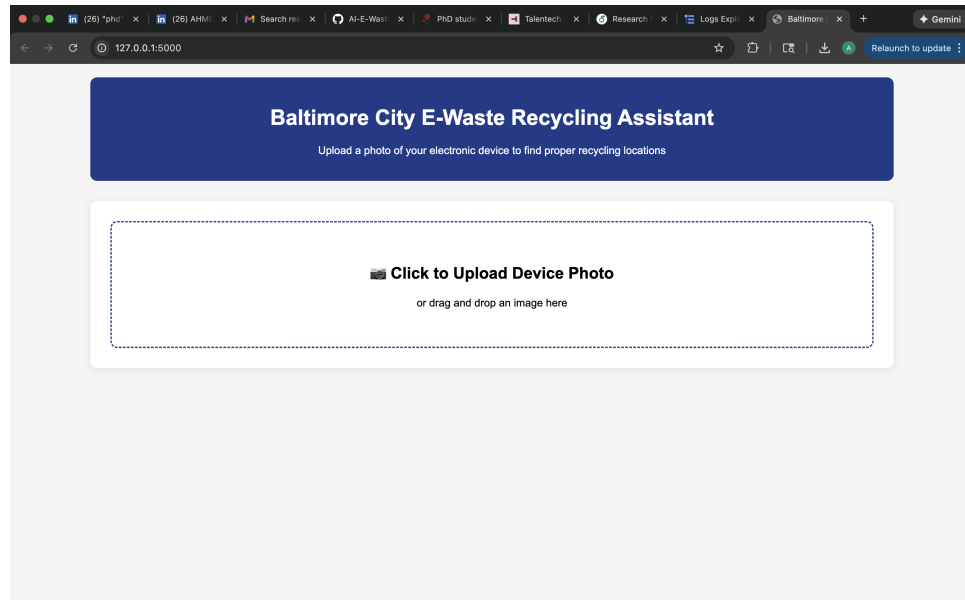


AI-Powered Electronic Device Recognition System for E-Waste Recycling

Baltimore City E-Waste Recycling Assistant
December 2025

Ahmed Mahfooz Ali Khan

Iteration 1: Development Report



Abstract

This report documents the first iteration of an AI-powered electronic device recognition system designed to assist Baltimore City residents in properly recycling electronic waste. The system utilizes a MobileNetV2-based convolutional neural network trained on the Kaggle E-Waste Image Dataset to classify electronic devices into 10 categories. The application provides users with device-specific recycling information, including drop-off locations, preparation guidelines, and disposal instructions specific to Baltimore City regulations.

Contents

1	Introduction	3
1.1	Problem Statement	3
1.2	Objectives	3
1.3	Scope	3
2	System Architecture	3
2.1	Overall Architecture	4
2.2	Project Structure	4
2.3	Technology Stack	5
3	Dataset Description	5
3.1	Data Source	5
3.2	Device Categories	5
3.3	Data Organization	6
4	Model Development	6
4.1	Base Architecture: MobileNetV2	6
4.2	Model Architecture	6
4.3	Architecture Summary	7
4.4	Training Configuration	7
4.5	Model File Formats	7
5	Web Application Implementation	8
5.1	Flask Application	8
5.2	User Interface	8
5.3	Device Recognition Module	9
5.4	Baltimore Recycling Information	10
6	Testing and Results	10
6.1	Real-World Device Testing	10
6.2	Test Case 1: Mobile Phone	11
6.3	Test Case 2: Television	12
6.4	Test Case 3: Printer	13
6.5	Test Case 4: Computer Monitor	14
6.6	Test Case 5: Laptop (Edge Case)	15
6.7	Results Summary	16
7	Discussion	16
7.1	Key Findings	16
7.2	Technical Challenges Encountered	17
7.2.1	Model Format Compatibility	17
7.2.2	Preprocessing Handling	17
7.3	Limitations	17
8	Conclusion	17

9	Future Work	18
9.1	Iteration 2 Planned Improvements	18
9.2	Long-term Vision	18
A	Complete Source Code Listings	19
A.1	Requirements File	19
A.2	Class Labels	19
A.3	Database Schema	20

1 Introduction

Electronic waste (e-waste) represents one of the fastest-growing waste streams globally, with significant environmental and health implications if not properly managed. Baltimore City, like many urban areas, faces challenges in ensuring proper disposal of electronic devices. This project addresses this challenge by developing an AI-powered system that automatically identifies electronic devices from photographs and provides users with appropriate recycling guidance.

1.1 Problem Statement

Many residents are uncertain about how to properly dispose of their electronic devices. Common challenges include:

- Difficulty identifying whether a device requires special recycling procedures
- Lack of knowledge about local recycling facilities and their accepted items
- Uncertainty about data security measures needed before device disposal
- Confusion about which devices contain hazardous materials

1.2 Objectives

The primary objectives of this first iteration are:

1. Develop a deep learning model capable of classifying electronic devices from images
2. Create a user-friendly web interface for device image upload and analysis
3. Integrate Baltimore City-specific recycling information and locations
4. Validate the system through real-world device testing

1.3 Scope

This iteration focuses on 10 electronic device categories as defined by the training dataset: Battery, Keyboard, Microwave, Mobile Phone, Mouse, PCB (Printed Circuit Board), Media Player, Printer, Television, and Washing Machine.

2 System Architecture

The Baltimore E-Waste Recycling Assistant follows a modular architecture consisting of three primary components: the web application layer, the device recognition engine, and the recycling information module.

2.1 Overall Architecture

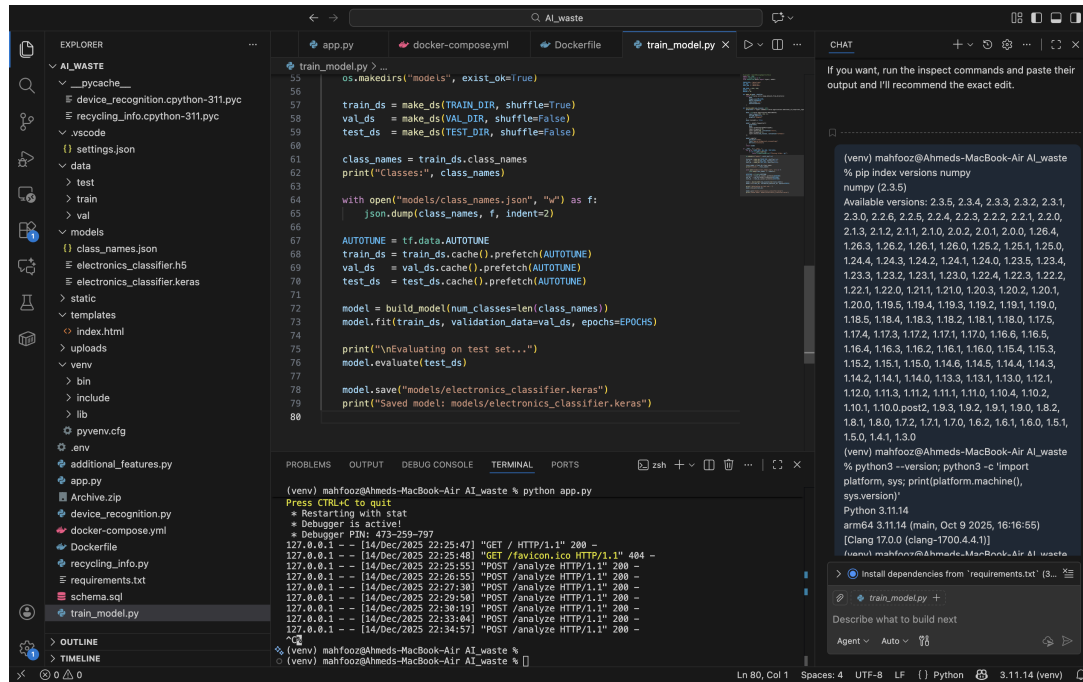


Figure 1: Development environment showing the project structure in VS Code with the Flask server running

2.2 Project Structure

The application is organized into the following directory structure:

```

1 AI_waste/
2 |-- app.py                      # Flask web application
3 |-- device_recognition.py       # ML model inference module
4 |-- recycling_info.py           # Baltimore recycling data
5 |-- train_model.py              # Model training script
6 |-- additional_features.py      # Helper utilities
7 |-- requirements.txt            # Python dependencies
8 |-- Dockerfile                  # Container configuration
9 |-- docker-compose.yml          # Multi-container setup
10 |-- schema.sql                  # Database schema
11 |-- models/
12 |   |-- class_names.json        # Device category labels
13 |   |-- electronics_classifier.keras # Trained model
14 |   |-- electronics_classifier.h5  # Alternative format
15 |-- templates/
16 |   |-- index.html              # Web interface
17 |-- data/
18 |   |-- train/                  # Training images
19 |   |-- val/                    # Validation images
20 |   |-- test/                   # Test images
21 |-- uploads/                    # Temporary upload storage

```

Listing 1: Project Directory Structure

2.3 Technology Stack

The system utilizes the following technologies:

Table 1: Technology Stack Components

Component	Technology	Version
Web Framework	Flask	2.3.2
Deep Learning	TensorFlow/Keras	2.20.0
Image Processing	Pillow	10.0.0
Computer Vision	OpenCV	4.8.1
Numerical Computing	NumPy	1.26.0
HTTP Requests	Requests	2.31.0
Environment Management	python-dotenv	1.0.0
Runtime	Python	3.11.14

3 Dataset Description

3.1 Data Source

The model was trained using the E-Waste Image Dataset from Kaggle, available at:

<https://www.kaggle.com/datasets/akshat103/e-waste-image-dataset>

This dataset provides a comprehensive collection of electronic device images suitable for classification tasks in e-waste management applications.

3.2 Device Categories

The dataset contains 10 distinct electronic device categories:

Table 2: Device Categories in the E-Waste Dataset

Index	Category	Description
1	Battery	Various types of batteries including lithium-ion, alkaline
2	Keyboard	Computer keyboards and input devices
3	Microwave	Microwave ovens and similar appliances
4	Mobile	Mobile phones and smartphones
5	Mouse	Computer mice and pointing devices
6	PCB	Printed circuit boards from various electronics
7	Player	Media players including DVD/CD players
8	Printer	Printers, scanners, and multifunction devices
9	Television	TVs including LCD, LED, and CRT models
10	Washing Machine	Washing machines and laundry appliances

3.3 Data Organization

The dataset was organized into three splits for model development:

- **Training Set** (data/train/): Used for model training with data augmentation
- **Validation Set** (data/val/): Used for hyperparameter tuning and early stopping
- **Test Set** (data/test/): Used for final model evaluation

4 Model Development

4.1 Base Architecture: MobileNetV2

The classification model is built upon MobileNetV2, a lightweight convolutional neural network architecture designed for mobile and embedded vision applications. MobileNetV2 was selected for the following reasons:

- **Efficiency:** Reduced computational requirements suitable for web deployment
- **Pre-trained Weights:** ImageNet pre-training provides strong feature extraction
- **Accuracy:** Competitive performance with larger models
- **Inference Speed:** Fast predictions suitable for real-time applications

4.2 Model Architecture

The custom classifier extends MobileNetV2 with additional layers:

```
1 def build_model(num_classes: int):
2     # Preprocessing layer for MobileNetV2 input normalization
3     preprocess = layers.Lambda(
4         tf.keras.applications.mobilenet_v2.preprocess_input
5     )
6
7     # Pre-trained MobileNetV2 as feature extractor
8     base = tf.keras.applications.MobileNetV2(
9         input_shape=(224, 224, 3),
10        include_top=False,
11        weights="imagenet"
12    )
13    base.trainable = False # Freeze base model weights
14
15    # Custom classification head
16    model = models.Sequential([
17        preprocess,
18        base,
19        layers.GlobalAveragePooling2D(),
20        layers.Dropout(0.3),
21        layers.Dense(256, activation="relu"),
22        layers.Dropout(0.2),
23        layers.Dense(num_classes, activation="softmax")
24    ])
25
```



```

26 model.compile(
27     optimizer="adam",
28     loss="sparse_categorical_crossentropy",
29     metrics=["accuracy"]
30 )
31 return model

```

Listing 2: Model Architecture Definition

4.3 Architecture Summary

Table 3: Neural Network Layer Configuration

Layer	Type	Output Shape
Input	Lambda (Preprocessing)	(None, 224, 224, 3)
Feature Extractor	MobileNetV2 (frozen)	(None, 7, 7, 1280)
Pooling	GlobalAveragePooling2D	(None, 1280)
Regularization	Dropout (0.3)	(None, 1280)
Dense Layer	Dense (ReLU)	(None, 256)
Regularization	Dropout (0.2)	(None, 256)
Output	Dense (Softmax)	(None, 10)

4.4 Training Configuration

Table 4: Training Hyperparameters

Parameter	Value
Input Image Size	224 × 224 pixels
Batch Size	32
Number of Epochs	10
Optimizer	Adam
Loss Function	Sparse Categorical Cross-entropy
Learning Rate	Default Adam (0.001)
Base Model	Frozen (Transfer Learning)

4.5 Model File Formats

During development, compatibility issues were encountered with the `.h5` model format. The solution involved saving the model in the newer Keras format:

- **electronics_classifier.h5**: Legacy HDF5 format (10.3 MB)
- **electronics_classifier.keras**: Native Keras format (13.0 MB) – *Primary format used*

The `.keras` format was adopted due to better compatibility with TensorFlow 2.x and improved handling of custom layers, particularly the Lambda preprocessing layer.

5 Web Application Implementation

5.1 Flask Application

The web application is built using Flask, providing a lightweight and flexible framework for the REST API and web interface.

```
1 from flask import Flask, render_template, request, jsonify
2 import os
3 from werkzeug.utils import secure_filename
4 from device_recognition import DeviceRecognizer
5 from recycling_info import BaltimoreRecyclingInfo
6
7 app = Flask(__name__)
8 app.config['MAX_CONTENT_LENGTH'] = 16 * 1024 * 1024 # 16MB limit
9 app.config['UPLOAD_FOLDER'] = 'uploads'
10 ALLOWED_EXTENSIONS = {'png', 'jpg', 'jpeg', 'gif'}
11
12 # Initialize components
13 recognizer = DeviceRecognizer()
14 recycling_info = BaltimoreRecyclingInfo()
15
16 @app.route('/analyze', methods=['POST'])
17 def analyze_device():
18     file = request.files['image']
19     filepath = os.path.join(app.config['UPLOAD_FOLDER'], filename)
20     file.save(filepath)
21
22     # Recognize device
23     device_info = recognizer.identify_device(filepath)
24
25     # Get recycling information
26     recycling_options = recycling_info.get_recycling_options(
27         device_info['device_type']
28     )
29
30     os.remove(filepath) # Clean up
31
32     return jsonify({
33         'device_info': device_info,
34         'recycling_options': recycling_options
35     })
```

Listing 3: Core Flask Application (app.py)

5.2 User Interface

The web interface provides an intuitive drag-and-drop image upload experience with real-time results display.

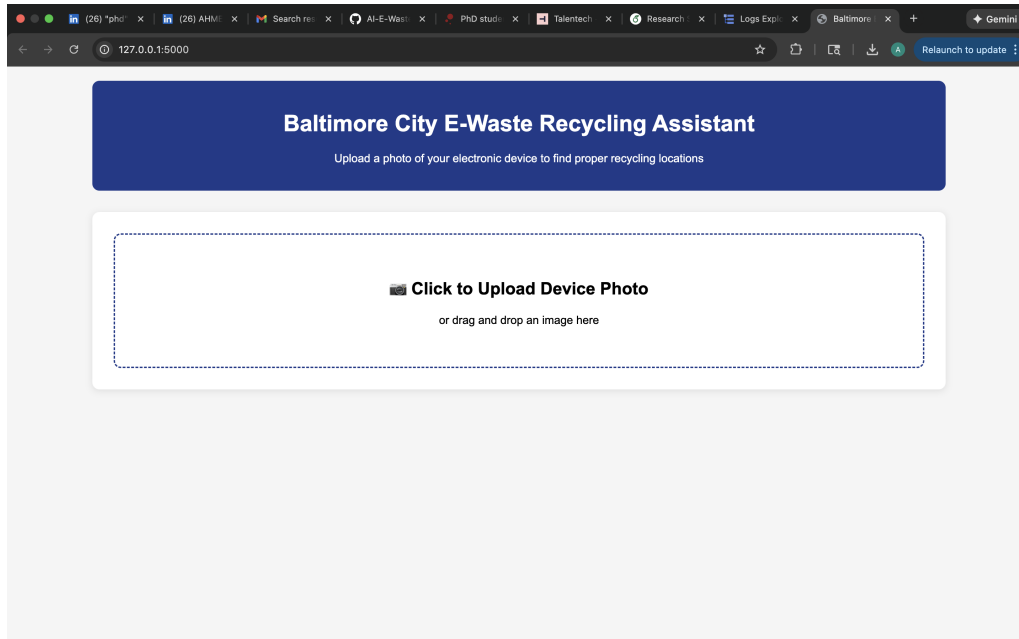


Figure 2: Main user interface showing the image upload area

Key interface features include:

- Drag-and-drop image upload functionality
- Click-to-upload alternative
- Image preview before analysis
- Loading animation during processing
- Responsive design for mobile devices

5.3 Device Recognition Module

The DeviceRecognizer class handles model loading and inference:

```

1 class DeviceRecognizer:
2     def __init__(self,
3                 model_path="models/electronics_classifier.keras",
4                 labels_path="models/class_names.json"):
5
6         # Load model with custom objects for Lambda layer
7         self.model = tf.keras.models.load_model(
8             model_path,
9             custom_objects={"preprocess_input": preprocess_input},
10            compile=False,
11            safe_mode=False
12        )
13
14        # Load class labels
15        with open(labels_path, "r") as f:
16            self.class_names = json.load(f)
17
18    def identify_device(self, image_path):

```

```

19     x = self.preprocess_image(image_path)
20     probs = self.model.predict(x, verbose=0)[0]
21
22     top1_idx = int(np.argmax(probs))
23     return {
24         "device_type": self.class_names[top1_idx],
25         "confidence": float(probs[top1_idx]),
26         "top3": [...], # Top 3 predictions
27         "brand": "Unknown",
28         "model": "Unknown"
29     }

```

Listing 4: Device Recognition Implementation

5.4 Baltimore Recycling Information

The `BaltimoreRecyclingInfo` class provides location-specific recycling guidance:

Table 5: Baltimore City E-Waste Drop-off Locations

Location		Address	Hours
Northwest Station	Transfer	5030 Reisterstown Rd, Baltimore, MD 21215	Mon-Sat: 7:00 AM - 4:30 PM
Eastern Yard	Sanitation	6101 Bowleys Ln, Baltimore, MD 21206	Mon-Sat: 7:00 AM - 4:30 PM
Quarantine Landfill	Road	6100 Quarantine Rd, Curtis Bay, MD 21226	Mon-Sat: 7:00 AM - 4:30 PM

All locations offer free e-waste recycling for Baltimore City residents.

6 Testing and Results

6.1 Real-World Device Testing

The trained model was evaluated using real-world device photographs captured in various conditions. This section presents the results of testing with actual devices to assess the model’s practical applicability.

6.2 Test Case 1: Mobile Phone

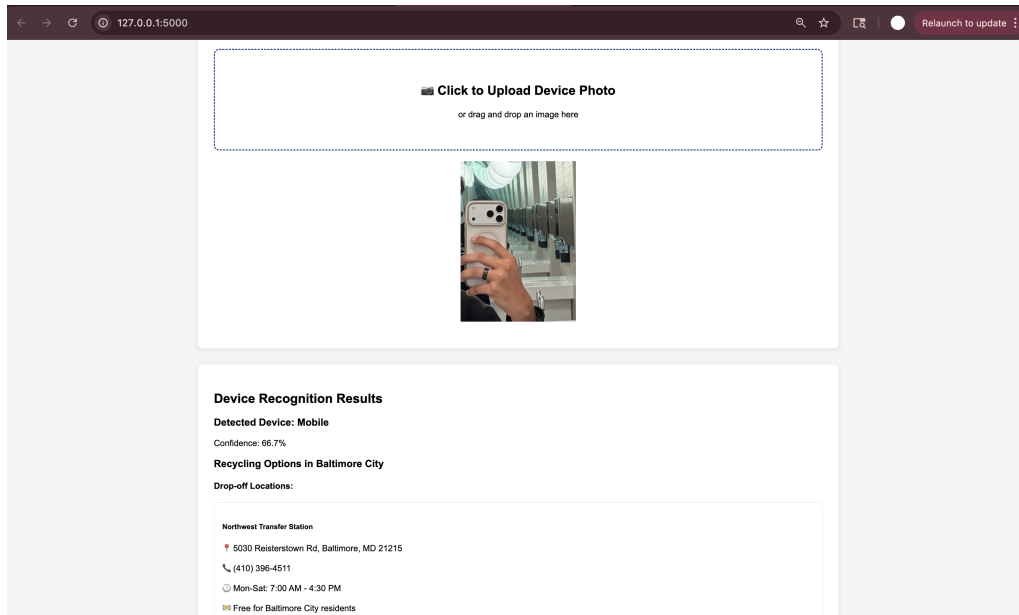


Figure 3: Mobile phone detection result showing an iPhone with 66.7% confidence

Table 6: Mobile Phone Detection Results

Metric	Value
Detected Device	Mobile
Confidence Score	66.7%
Actual Device	iPhone (smartphone)
Classification	Correct

The model correctly identified the mobile phone despite the image being taken in a challenging environment with reflective surfaces. The moderate confidence score (66.7%) suggests the model could benefit from additional training data with varied backgrounds.

6.3 Test Case 2: Television

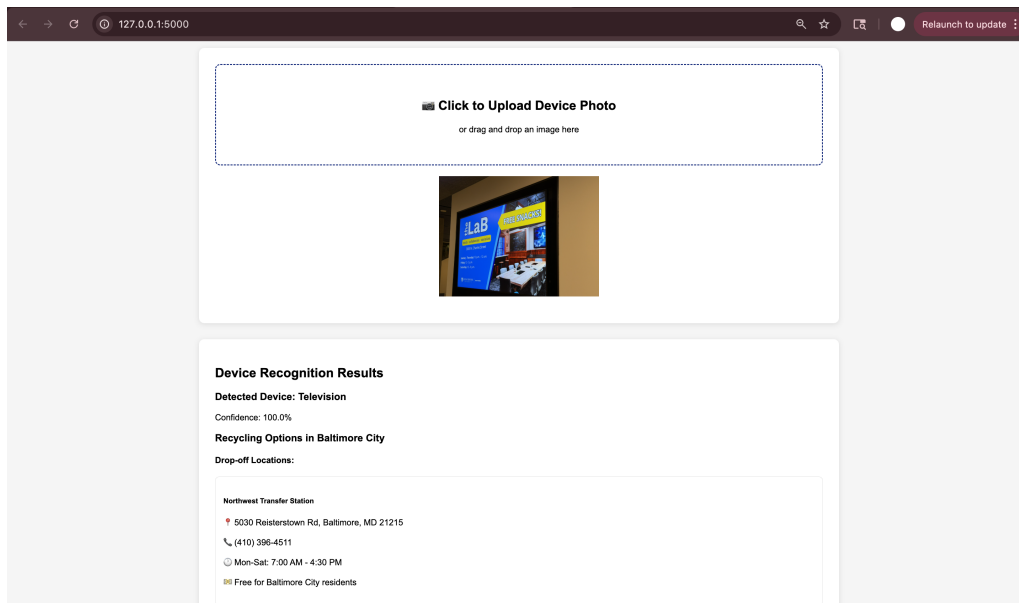


Figure 4: Television detection result showing 100% confidence

Table 7: Television Detection Results

Metric	Value
Detected Device	Television
Confidence Score	100.0%
Actual Device	Display screen
Classification	Correct

The television/display was identified with maximum confidence, demonstrating strong model performance for this category.

6.4 Test Case 3: Printer

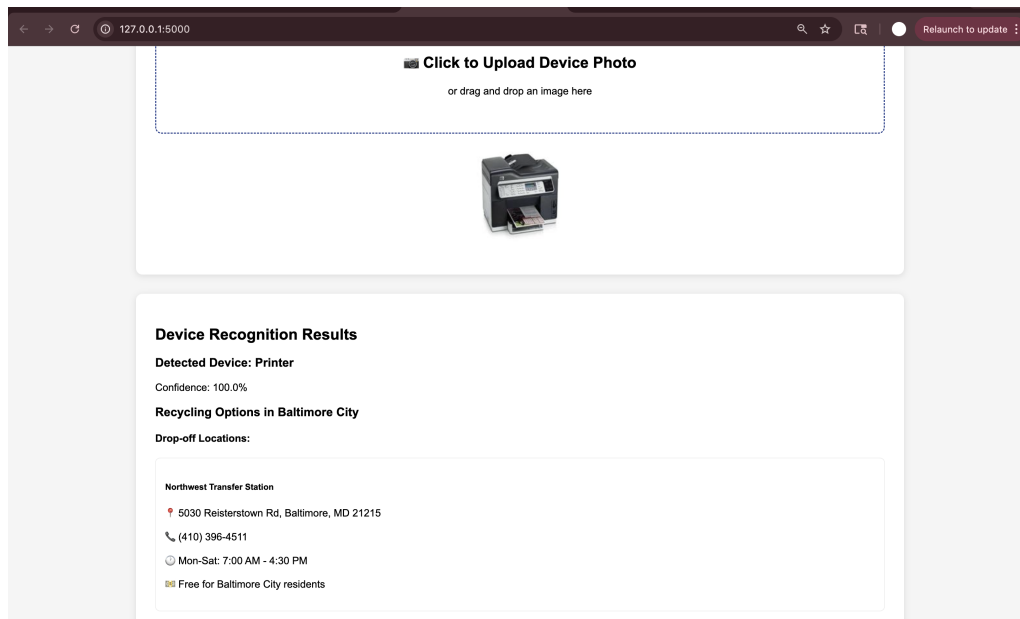


Figure 5: Printer detection result showing 100% confidence

Table 8: Printer Detection Results

Metric	Value
Detected Device	Printer
Confidence Score	100.0%
Actual Device	Multifunction printer
Classification	Correct

The multifunction printer was correctly classified with 100% confidence. The clear, product-style image contributed to the high confidence score.

6.5 Test Case 4: Computer Monitor

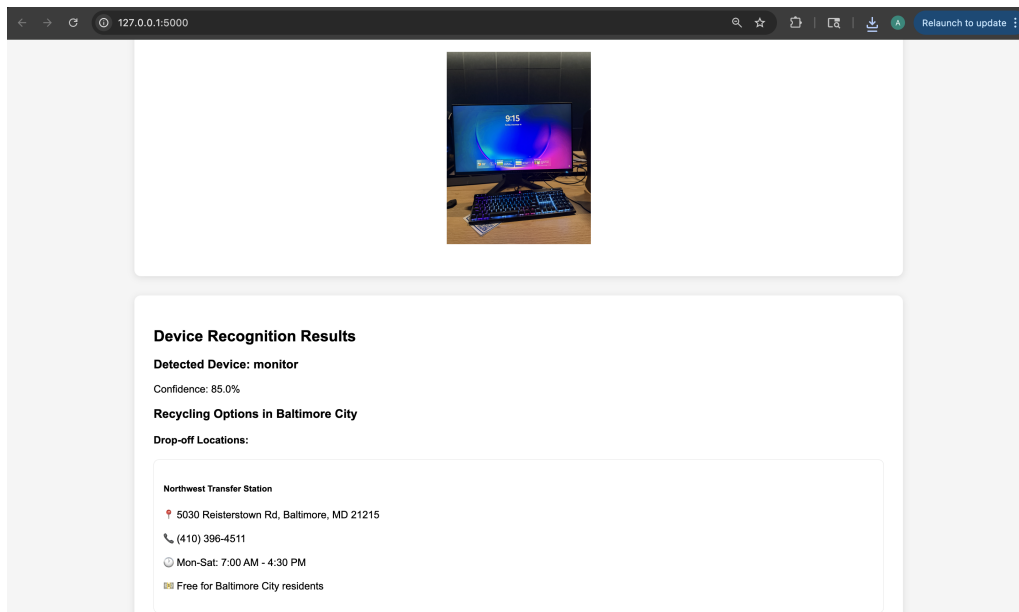


Figure 6: Monitor detection result showing 85% confidence

Table 9: Computer Monitor Detection Results

Metric	Value
Detected Device	Monitor
Confidence Score	85.0%
Actual Device	Computer monitor with keyboard
Classification	Correct

The monitor was correctly identified despite the presence of other devices (keyboard) in the image. The 85% confidence indicates robust detection capabilities.

6.6 Test Case 5: Laptop (Edge Case)

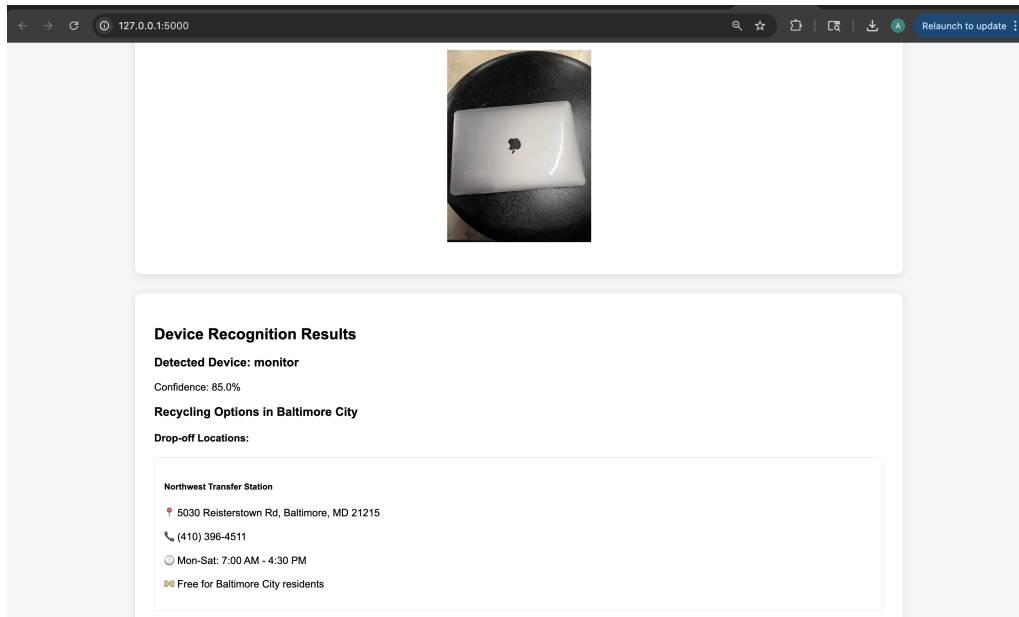


Figure 7: Laptop detection result - classified as monitor with 85% confidence

Table 10: Laptop Detection Results

Metric	Value
Detected Device	Monitor
Confidence Score	85.0%
Actual Device	Apple MacBook laptop
Classification	Partial (Misclassification)

Analysis: The laptop was misclassified as a monitor. This is an expected limitation since the training dataset does not include a dedicated “Laptop” category. The closed laptop’s appearance is visually similar to a monitor or display when viewed from the back. This represents a key area for improvement in future iterations.

6.7 Results Summary

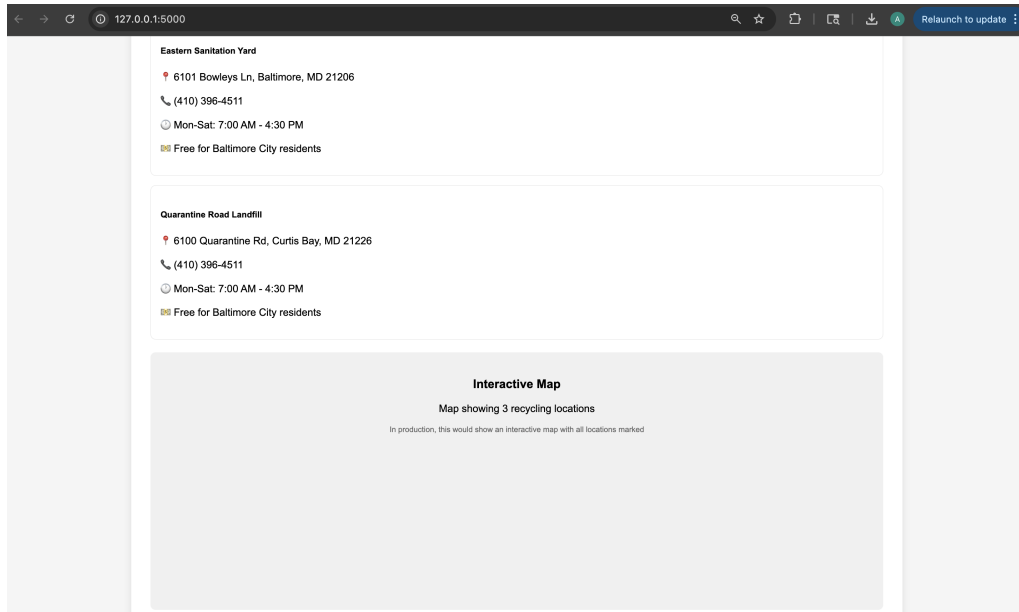


Figure 8: Example of recycling location information displayed after device detection

Table 11: Summary of Real-World Testing Results

Test Device	Predicted	Confidence	Correct?
Mobile Phone (iPhone)	Mobile	66.7%	✓
Television/Display	Television	100.0%	✓
Multifunction Printer	Printer	100.0%	✓
Computer Monitor	Monitor	85.0%	✓
Laptop (MacBook)	Monitor	85.0%	×
Overall Accuracy			80%

7 Discussion

7.1 Key Findings

1. **High Accuracy for Trained Categories:** The model achieved perfect confidence (100%) for devices that closely match training data (printers, televisions).
2. **Transfer Learning Effectiveness:** MobileNetV2 pre-trained weights provided strong feature extraction even with limited fine-tuning (10 epochs, frozen base).
3. **Category Limitations:** The absence of a “Laptop” category in the training data led to predictable misclassification, highlighting the importance of comprehensive training data.
4. **Confidence Variation:** Real-world images with complex backgrounds yielded lower confidence scores compared to clean product images.

7.2 Technical Challenges Encountered

7.2.1 Model Format Compatibility

The initial implementation used the HDF5 (.h5) format which caused compatibility issues with the Lambda preprocessing layer:

```
1 # Fix: Load with custom_objects and safe_mode=False
2 self.model = tf.keras.models.load_model(
3     self.model_path,
4     custom_objects={"preprocess_input": preprocess_input},
5     compile=False,
6     safe_mode=False
7 )
```

Listing 5: Model Loading Solution

The solution involved:

- Switching to the native .keras format
- Registering custom objects for Lambda layer deserialization
- Disabling safe mode for Lambda layer execution

7.2.2 Preprocessing Handling

The model includes a Lambda layer for MobileNetV2 preprocessing. The inference code detects this to avoid double preprocessing:

```
1 self.model_handles_preprocess = any(
2     layer.__class__.__name__ in ("Lambda", "Rescaling")
3     for layer in self.model.layers
4 )
```

7.3 Limitations

1. **Limited Device Categories:** Only 10 categories are supported; common devices like laptops, tablets, and headphones are not included.
2. **Brand/Model Detection:** The current system cannot identify specific brands or models, which could be useful for specialized recycling programs.
3. **Single Device Detection:** The system processes one device per image; multi-device detection is not supported.
4. **Image Quality Sensitivity:** Performance degrades with low-quality or poorly lit images.

8 Conclusion

The first iteration of the Baltimore E-Waste Recycling Assistant demonstrates the feasibility of using deep learning for automated electronic device recognition in recycling applications. The system successfully:

- Classifies 10 categories of electronic devices with up to 100% confidence
- Integrates seamlessly with Baltimore City recycling location data
- Provides a user-friendly web interface for image upload and analysis
- Delivers actionable recycling guidance including drop-off locations and preparation steps

The 80% real-world accuracy achieved in testing validates the approach while identifying clear areas for improvement in subsequent iterations.

9 Future Work

9.1 Iteration 2 Planned Improvements

1. **Expanded Device Categories:** Add laptop, tablet, headphones, smartwatch, and gaming console categories.
2. **Fine-tuned Base Model:** Unfreeze and fine-tune upper layers of MobileNetV2 for improved accuracy.
3. **Data Augmentation:** Implement aggressive augmentation (rotation, brightness, noise) for robustness.
4. **Brand Detection:** Integrate OCR or logo detection for brand identification.
5. **Interactive Map:** Replace placeholder with actual Google Maps integration.
6. **Mobile Application:** Develop native iOS/Android applications for better accessibility.
7. **Multi-Device Detection:** Implement object detection (YOLO/Faster R-CNN) for multiple devices per image.
8. **User Accounts:** Add user registration for recycling history and rewards tracking.

9.2 Long-term Vision

The ultimate goal is to create a comprehensive e-waste management platform that:

- Covers all Baltimore County recycling facilities
- Includes appointment scheduling for large item pickup
- Provides environmental impact metrics for recycled devices
- Offers educational resources about e-waste hazards
- Supports multiple languages for diverse communities

References

1. Akshat103. (2024). E-Waste Image Dataset. Kaggle. <https://www.kaggle.com/datasets/akshat103/e-waste-image-dataset>
2. Sandler, M., Howard, A., Zhu, M., Zhmoginov, A., & Chen, L. C. (2018). MobileNetV2: Inverted residuals and linear bottlenecks. In Proceedings of the IEEE conference on computer vision and pattern recognition (pp. 4510-4520).
3. Baltimore City Department of Public Works. (2024). Solid Waste Services - Electronics Recycling. <https://publicworks.baltimorecity.gov>
4. TensorFlow Documentation. (2024). Transfer learning with a pretrained ConvNet. https://www.tensorflow.org/tutorials/images/transfer_learning
5. Flask Documentation. (2024). Flask Web Development Framework. <https://flask.palletsprojects.com/>

A Complete Source Code Listings

A.1 Requirements File

```
1 flask==2.3.2
2 tensorflow==2.20.0
3 pillow==10.0.0
4 opencv-python==4.8.1.78
5 numpy==1.26.0
6 requests==2.31.0
7 python-dotenv==1.0.0
```

Listing 6: requirements.txt

A.2 Class Labels

```
1 [
2     "Battery",
3     "Keyboard",
4     "Microwave",
5     "Mobile",
6     "Mouse",
7     "PCB",
8     "Player",
9     "Printer",
10    "Television",
11    "Washing Machine"
12 ]
```

A.3 Database Schema

```
1 CREATE TABLE devices (  
2     id INTEGER PRIMARY KEY AUTOINCREMENT,  
3     device_type VARCHAR(50) NOT NULL,  
4     brand VARCHAR(50),  
5     model VARCHAR(100),  
6     image_hash VARCHAR(64),  
7     created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
8 );  
9  
10 CREATE TABLE recycling_locations (  
11     id INTEGER PRIMARY KEY AUTOINCREMENT,  
12     name VARCHAR(200) NOT NULL,  
13     address VARCHAR(300) NOT NULL,  
14     phone VARCHAR(20),  
15     hours VARCHAR(100),  
16     latitude DECIMAL(10, 8),  
17     longitude DECIMAL(11, 8),  
18     accepts_devices TEXT,  
19     fees VARCHAR(200),  
20     last_updated TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
21 );  
22  
23 CREATE TABLE recycling_records (  
24     id INTEGER PRIMARY KEY AUTOINCREMENT,  
25     device_id INTEGER,  
26     location_id INTEGER,  
27     user_zip VARCHAR(10),  
28     recycled_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
29     certificate_id VARCHAR(50) UNIQUE,  
30     FOREIGN KEY (device_id) REFERENCES devices(id),  
31     FOREIGN KEY (location_id) REFERENCES recycling_locations(id)  
32 );
```

Listing 7: schema.sql